

Simple Circuit Simulator

Milestone 5 Report - Class Description and Architecture Documentation

1. Project Overview

The Simple Circuit Simulator is a Python-based application designed to model and analyze direct current (DC) electrical circuits. The simulator enables users to construct circuits containing voltage sources, resistors, and constant impedance loads connected across multiple electrical nodes (buses). The primary functionality is to calculate nodal voltages at each bus and determine the circuit current flowing through the system using Kirchhoff's Voltage Law (KVL) and conductance-based analysis techniques.

Purpose

The simulator serves educational and analytical purposes by providing a computational framework for understanding and solving basic circuit problems. It demonstrates fundamental electrical principles including Ohm's Law, power-voltage relationships, and nodal analysis methods.

Key Features

- **Component Modeling:** Accurate representation of voltage sources, resistors, and constant impedance loads
- **Nodal Analysis:** Calculation of voltage at each electrical node (bus) in the circuit

- **Current Calculation:** Determination of circuit current based on component properties
- **Extensible Architecture:** Object-oriented design allowing easy addition of new component types

Real-World Applications

The techniques employed in this simulator are foundational to power systems analysis, electrical circuit design, and automated power flow studies used in modern electrical grids and industrial applications.

2. Class Architecture and Descriptions

2.1 Bus Class (Milestone 2)

Purpose: Represents an electrical node within the circuit where voltage is measured and tracked.

Attributes:

Attribute	Type	Description
<code>name</code>	<code>str</code>	User-provided identifier for the bus node (e.g., "A", "B")
<code>v</code>	<code>float</code>	Voltage at the bus in volts. Initialized based on source connections or updated during power flow calculation

Methods:

- `set_bus_v(bus_v: float) -> void` : Updates the voltage value at the bus to the specified value

Implementation Notes:

- Bus objects serve as reference points in the circuit where voltage is measured
- For buses connected to a voltage source, the voltage is set when the source is created
- For buses not directly connected to a voltage source, the voltage is calculated during power flow analysis
- The `set_bus_v()` method is the primary interface for voltage updates

2.2 Resistor Class (Milestone 2)

Purpose: Models a linear resistive element between two electrical nodes, characterized by resistance and conductance values.

Attributes:

Attribute	Type	Description
<code>name</code>	<code>str</code>	User-provided identifier for the resistor element (e.g., "Rab")
<code>bus1</code>	<code>str</code>	Name of the first connected bus node
<code>bus2</code>	<code>str</code>	Name of the second connected bus node
<code>r</code>	<code>float</code>	Resistance value in ohms (Ω), provided by the user
<code>g</code>	<code>float</code>	Conductance value in siemens (S), calculated internally from resistance

Methods:

- `calc_g() -> void` : Calculates conductance as the reciprocal of resistance

Mathematical Foundation:

$$g = 1/r$$

where g is conductance in siemens and r is resistance in ohms.

2.3 Load Class (Milestone 2)

Purpose: Models a constant impedance load element, typically representing a connected electrical device with fixed power consumption characteristics.

Attributes:

Attribute	Type	Description
name	str	User-provided identifier for the load element (e.g., "Lb")
bus1	str	Name of the bus node where the load is connected (single-terminal connection)
p	float	Nominal power consumption of the load in watts (W)
v	float	Nominal voltage rating of the load in volts (V)
r	float	Equivalent resistance calculated from nominal power and voltage conditions
g	float	Equivalent conductance calculated from the equivalent resistance

Methods:

- `calc_g()` -> void : Calculates conductance from the equivalent resistance

Mathematical Foundation:

$$r = v^2/p$$
$$g = 1/r = p/v^2$$

2.4 VSource Class (Milestone 2)

Purpose: Represents an ideal voltage source element that maintains a specified voltage at a connected bus node.

Attributes:

Attribute	Type	Description
<code>name</code>	<code>str</code>	User-provided identifier for the voltage source (e.g., "Va")
<code>bus1</code>	<code>str</code>	Name of the bus node where the voltage source is connected
<code>v</code>	<code>float</code>	Voltage supplied by the source in volts (V)

Methods:

None - VSource is a simple data container with no calculation methods.

2.5 Circuit Class (Milestone 3)

Purpose: Serves as the central container and manager for all circuit components, coordinating their interactions and providing the interface for circuit configuration and analysis.

Attributes:

Attribute	Type	Description
<code>name</code>	<code>str</code>	User-provided identifier for the circuit (e.g., "Simple DC Circuit")

Attribute	Type	Description
buses	dict[str, Bus]	Dictionary mapping bus names (strings) to Bus objects
resistors	dict[str, Resistor]	Dictionary mapping resistor names to Resistor objects
loads	dict[str, Load]	Dictionary mapping load names to Load objects
vsource	VSource	Single voltage source object for the circuit
i	float	Circuit current in amperes (A), updated during power flow calculation

Methods:

- `add_bus(bus: str) -> void` : Creates and adds a new Bus object to the circuit
- `add_resistor_element(name: str, bus1: str, bus2: str, r: float) -> void` : Creates and adds a Resistor between two buses
- `add_load_element(name: str, bus1: str, p: float, v: float) -> void` : Creates and adds a Load element to a bus
- `add_vsource_element(name: str, bus1: str, v: float) -> void` : Creates and adds a VSource to a bus
- `set_i(i: float) -> void` : Updates the circuit current attribute
- `print_nodal_voltage() -> void` : Displays the voltage at all buses
- `print_circuit_current() -> void` : Displays the circuit current

2.6 Solution Class (Milestone 4)

Purpose: Implements the power flow algorithm to solve the circuit and determine

nodal voltages and circuit current.

Attributes:

Attribute	Type	Description
<code>circuit</code>	<code>Circuit</code>	Reference to the <code>Circuit</code> object to be analyzed

Methods:

- `do_power_flow() -> void` : Executes the circuit analysis algorithm to calculate bus voltages and circuit current

Algorithm Overview:

1. **Conductance Calculation:** Calculates conductance for all resistors and loads using their respective `calc_g()` methods
2. **Parallel Conductance:** Determines the total conductance seen from bus B by summing the conductances of the series resistor and load
3. **Voltage Division:** Uses voltage divider principles to calculate the voltage at bus B
4. **Current Calculation:** Determines circuit current using the voltage drop across the series resistor and its conductance

3. Class Relationships and Architecture

Composition Hierarchy

```
Solution (1)
├─ Circuit (1)
│   ├─ Bus (many)
│   ├─ Resistor (many)
│   └─ Load (many)
```

 vSource (1)

Element Initialization Flow

1. User creates a Circuit object
2. User calls `add_bus()` to create buses
3. User calls `add_vsource_element()` to set voltage reference
4. User calls `add_resistor_element()` and `add_load_element()` to define circuit topology
5. Solution object is created with the configured Circuit
6. `do_power_flow()` is called to calculate results

4. Key Algorithms and Equations

4.1 Ohm's Law

$$V = I \cdot R$$

$$I = V \cdot G$$

4.2 Power-Voltage Relationship

$$P = V \cdot I = V^2/R = V^2 \cdot G$$

For constant impedance loads:

$$R = V^2/P$$

4.3 Conductance Calculation

$$G = I/R$$

For loads:

$$G = P/V^2$$

4.4 Kirchhoff's Voltage Law (KVL)

The sum of voltages around any closed loop in a circuit equals zero:

$$\Sigma V = 0$$

4.5 Nodal Analysis

For the simple circuit configuration, the voltage at bus B is determined by:

$$V_B = V_A \cdot (G_{load} / (G_{series} + G_{load}))$$

4.6 Circuit Current

$$I = (V_A - V_B) \cdot G_{series}$$

5. Example Case Study

Problem Definition

Circuit Configuration:

- Voltage Source: 100 V at bus A
- Series Resistor: 5 Ω connecting bus A to bus B

- Load: 2000 W rated at 100 V connected to bus B

Known Values:

- $V_A = 100 \text{ V}$
- $R_{\text{series}} = 5 \text{ } \Omega$
- $P_{\text{load}} = 2000 \text{ W}$
- $V_{\text{load,nominal}} = 100 \text{ V}$

Solution Process

Step 1: Calculate Conductances

Load equivalent resistance:

$$R_{\text{load}} = V_{\text{load,nominal}}^2 / P_{\text{load}} = 100^2 / 2000 = 10000 / 2000 = 5 \text{ } \Omega$$

Load conductance:

$$G_{\text{load}} = 1 / R_{\text{load}} = 1 / 5 = 0.2 \text{ S}$$

Series resistor conductance:

$$G_{\text{series}} = 1 / R_{\text{series}} = 1 / 5 = 0.2 \text{ S}$$

Step 2: Calculate Bus B Voltage

Using voltage divider:

$$V_B = V_A \cdot (G_{\text{load}} / (G_{\text{series}} + G_{\text{load}}))$$
$$V_B = 100 \cdot (0.2 / (0.2 + 0.2)) = 100 \cdot (0.2 / 0.4) = 100 \cdot 0.5 = 50.0 \text{ V}$$

Step 3: Calculate Circuit Current

Using Ohm's Law:

$$I = (V_A - V_B) \cdot G_{series}$$

$$I = (100 - 50.0) \cdot 0.2 = 50.0 \cdot 0.2 = 10.0 \text{ A}$$

Expected Output

```
Bus A voltage = 100.0 V Bus B voltage = 50.0 V Circuit  
current = 10.0 A
```

Verification

Power consumed by load at operating voltage:

$$P_{actual} = V_B^2 / R_{load} = 50.0^2 / 5 = 2500 / 5 = 500 \text{ W}$$

Power dissipated in series resistor:

$$P_{series} = I^2 \cdot R_{series} = 10.0^2 \cdot 5 = 100 \cdot 5 = 500 \text{ W}$$

Power supplied by source:

$$P_{source} = V_A \cdot I = 100 \cdot 10.0 = 1000 \text{ W}$$

Verification: $P_{source} = P_{actual} + P_{series}$ (500 + 500 = 1000 W) ✓

6. Implementation Summary

Milestone 2 Deliverables

- **Bus Class:** Basic node representation with voltage storage and update capability

- **Resistor Class:** Two-terminal element with conductance calculation
- **Load Class:** Single-terminal constant impedance element with power-based conductance calculation
- **VSource Class:** Voltage reference element with fixed output voltage

Milestone 3 Deliverables

- **Circuit Class:** Component aggregator providing interface for circuit construction and result display

Milestone 4 Deliverables

- **Solution Class:** Power flow solver implementing nodal analysis algorithm

Design Patterns

- **Composition:** Solution contains Circuit, which contains all electrical elements
- **Dictionary-based Storage:** Components stored by name for efficient access
- **Separation of Concerns:** Each class handles a specific responsibility (modeling, management, solving)
- **Encapsulation:** Methods like `calc_g()` encapsulate internal calculation logic

7. Conclusion

The Simple Circuit Simulator demonstrates fundamental object-oriented design principles applied to electrical circuit analysis. By decomposing the problem into six distinct classes, each with well-defined responsibilities, the implementation achieves clarity, maintainability, and extensibility. The architecture supports educational objectives by making circuit analysis algorithms explicit and understandable, while the object-oriented structure provides a foundation for future enhancements and more complex circuit topologies.

The class hierarchy and inter-component relationships establish a clear separation between circuit modeling (Elements), circuit management (Circuit), and circuit analysis (Solution), enabling students to understand both the electrical principles and software engineering best practices.